# Initializing events and any relevent rate cut events

In [1]:
```
import pandas as pd
import yfinance as yf
import numpy as np
import statsmodels.api as sm
from scipy import stats

import warnings
warnings.filterwarnings('ignore')

# Creating a dataframe for the rate cuts
events = pd.DataFrame({
    "date": [
        "2001-01-03","2001-01-31","2001-03-20","2001-04-18","2001-05-1
        "2001-08-21","2001-09-17","2001-10-02","2001-11-06","2001-12-1
        "2002-11-06","2003-06-25",
        "2007-09-18","2007-10-31","2007-12-11",
        "2008-01-22","2008-01-30","2008-03-18","2008-04-30",
        "2008-10-08","2008-10-29","2008-12-16",
        "2019-08-01","2019-09-19","2019-10-31",
        "2020-03-03","2020-03-15",
        "2024-09-18","2024-11-07","2024-12-18",
        "2025-09-17","2025-10-29","2025-12-10"
    ],
    "cut_bps": [
        50,50,50,50,50,25,
        25,50,50,50,25,
        50,25,
        50,25,25,
        75,50,75,25,
        50,50,100,
        25,25,25,
        50,100,
        50,25,25,
        25,25,25
    ]
})

events["date"] = pd.to_datetime(events["date"], errors="coerce")

def size_bin(bps):
    if bps == 25:
        return "25bps"
    elif bps == 50:
        return "50bps"
    else:
```

```
        return "75bps+"

# Creating three bins
events["cut_bin"] = events["cut_bps"].apply(size_bin)


# Specifying significant financial events that have occured
events["crisis"] = 0

events.loc[
    (events["date"].dt.year <= 2003) |
    (events["date"].dt.year >= 2007) & (events["date"].dt.year <= 2009
    (events["date"].dt.year == 2020), "crisis"] = 1


# Specifying urgent rate cuts - occured outside the schedule FOMC
inter_meeting_dates = [
    "2001-01-03","2001-09-17",
    "2008-01-22","2008-10-08",
    "2020-03-03","2020-03-15"
]

events["inter_meeting"] = events["date"].isin(pd.to_datetime(inter_mee
```

# Pulling prices and dropping N/A rows if any

In [2]:
```
tickers = [
    "^BKX","KIE","VFH","FINX",
    "AMP","BX","MKL","MS","O","PLD","V","WFC","SPY"
]

start_date = "2000-01-01"
end_date = "2026-01-31"

prices = yf.download(
    tickers,
    start=start_date,
    end=end_date,
    auto_adjust=True
)["Close"]

# Dropping null rows
all_nan_rows = prices[prices.isna().all(axis=1)]

print(f"Dropping {len(all_nan_rows)} rows where all prices are NaN")
prices = prices.dropna(how="all").sort_index()
```

```
[**********************100%**********************]  13 of 13 completed
```

```
Dropping 0 rows where all prices are NaN
```

# Calculating Returns and Excess Returns

In [3]:
```python
# Calculating percent changes for each day
returns = np.log(prices / prices.shift(1))
returns = returns.dropna(how="all")

# Using for comparing relative returns
market = returns["SPY"]
asset_returns = returns.drop(columns="SPY")
excess_returns = asset_returns.sub(market, axis=0)
```

In [4]:
```python
# Using just in cases there was an event during a holiday or weekend -
def next_trading_day(date, trading_days):
    if date in trading_days:
        return date
    return trading_days[trading_days.searchsorted(date)]

trading_days = returns.index

events["event_date"] = events["date"].apply(
    lambda d: next_trading_day(d, trading_days)
)
```

# Compuuting the returns on 1d, 1w, 1m timeframe for the assets total return and relative return

In [5]:
```python
def compute_window(event, returns_df, window):
    if event not in prices.index:
        print("Could not find event date")
        return None

    # Getting price change at event date and the the duration
    start_loc = returns_df.index.get_loc(event)
    end_loc = start_loc + window

    # Making sure we don't exceed available data
    if end_loc >= len(returns_df):
        return None

    # Getting that time frame of price change
    window_slice = returns_df.iloc[start_loc:end_loc+1]

    # Suming log returns across window
```

```
    return window_slice.sum()

# New DF for the events and price change
event_results = []

for _, row in events.iterrows():

    event_date = row["event_date"]

    # Getting changes for each window for each asset
    day_ret_a   = compute_window(event_date, asset_returns, 0)
    week_ret_a  = compute_window(event_date, asset_returns, 5)
    month_ret_a = compute_window(event_date, asset_returns, 21)

    # Getting changes for each window for each asset relative return
    day_ret_e   = compute_window(event_date, excess_returns, 0)
    week_ret_e  = compute_window(event_date, excess_returns, 5)
    month_ret_e = compute_window(event_date, excess_returns, 21)

    if day_ret_a is None:
        continue

    # Adding to the new DF
    event_results.append({
        "event_date": event_date,
        "cut_bps": row["cut_bps"],
        "cut_bin": row["cut_bin"],
        "crisis": row["crisis"],
        "day_a": day_ret_a,
        "week_a": week_ret_a,
        "month_a": month_ret_a,
        "day_e": day_ret_e,
        "week_e": week_ret_e,
        "month_e": month_ret_e
    })

event_panel = pd.DataFrame(event_results)
# Formatting better
expanded_rows = []
for _, row in event_panel.iterrows():

    for asset in asset_returns.columns:

        expanded_rows.append({
            "event_date": row["event_date"],
            "asset": asset,
            "cut_bps": row["cut_bps"],
            "cut_bin": row["cut_bin"],
            "crisis": row["crisis"],
            "day_a": row["day_a"][asset],
            "week_a": row["week_a"][asset],
            "month_a": row["month_a"][asset],
```

```
            "day_e": row["day_e"][asset],
            "week_e": row["week_e"][asset],
            "month_e": row["month_e"][asset]
        })

# Will be used for later
event_panel_asset = pd.DataFrame(expanded_rows)

event_panel = pd.DataFrame(expanded_rows)

# Creating groupiing
event_level = (
    event_panel
    .groupby("event_date", as_index=False)
    .agg({
        "cut_bps": "first",
        "crisis": "first",
        "day_a": "mean",
        "week_a": "mean",
        "month_a": "mean",
        "day_e": "mean",
        "week_e": "mean",
        "month_e": "mean"
    })
)
```

In [6]:
```
# Sanity Check - making sure grouping is correct - 34 events and 9 dif
event_level.head()
event_level.shape
```

Out[6]: (34, 9)

# Printing correlation

In [7]:
```
corr_day_a   = event_level["cut_bps"].corr(event_level["day_a"])
corr_week_a  = event_level["cut_bps"].corr(event_level["week_a"])
corr_month_a = event_level["cut_bps"].corr(event_level["month_a"])

corr_day_e   = event_level["cut_bps"].corr(event_level["day_e"])
corr_week_e  = event_level["cut_bps"].corr(event_level["week_e"])
corr_month_e = event_level["cut_bps"].corr(event_level["month_e"])

print("Asset Returns")
print("Day  :", corr_day_a)
print("Week :", corr_week_a)
print("Month:", corr_month_a)

print("\nExcess Returns")
print("Day  :", corr_day_e)
print("Week :", corr_week_e)
```

```
print("Month:", corr_month_e)
```

```
Asset Returns
Day  : -0.030429593556819175
Week : -0.17358479117408887
Month: -0.047802347665123104

Excess Returns
Day  : 0.12769417965503177
Week : 0.027470760079725737
Month: -0.23883120961349452
```

# Seeing correlation and finding any relative information

Chatgpt did this section totally - originally did only month but wanted to see each timeframe

In [18]:
```
def summarize_window(df, window):
    ret = f"{window}_e"   # only excess returns

    tables = {}

    # Mean excess return by cut size and crisis
    tables["cut_crisis_mean"] = (
        df
        .groupby(["cut_bps", "crisis"])[ret]
        .mean()
        .unstack("crisis")
    )

    # Correlation with cut size
    tables["correlation"] = df[["cut_bps", ret]].corr()

    # Crisis vs non-crisis mean + count
    tables["crisis_summary"] = (
        df
        .groupby("crisis")[ret]
        .agg(["mean", "count"])
    )

    # Crisis × cut size detail
    tables["crisis_cut_detail"] = (
        df
        .groupby(["crisis", "cut_bps"])[ret]
        .mean()
    )

    return tables
```

```
day_tables   = summarize_window(event_level, "day")
week_tables  = summarize_window(event_level, "week")
month_tables = summarize_window(event_level, "month")
```

In [19]:
```
print("=== DAY: Mean Excess Returns by Cut & Crisis ===")
print(day_tables["cut_crisis_mean"], "\n")

print("=== DAY: Correlation with Cut Size ===")
print(day_tables["correlation"], "\n")

print("=== DAY: Crisis vs Non-Crisis ===")
print(day_tables["crisis_summary"], "\n")
```

```
=== DAY: Mean Excess Returns by Cut & Crisis ===
crisis          0         1
cut_bps
25      -0.005163 -0.000151
50      -0.001253 -0.000065
75            NaN  0.023034
100           NaN -0.006224

=== DAY: Correlation with Cut Size ===
           cut_bps     day_e
cut_bps   1.000000  0.127694
day_e     0.127694  1.000000

=== DAY: Crisis vs Non-Crisis ===
             mean  count
crisis
0      -0.004729      9
1       0.001266     25
```

# Interpetation (Day)

On the announcement day, rate cuts outside crisis periods are associated with negative excess returns, suggesting markets interpret them as signals of economic weakness. In contrast, crisis-period cuts generate neutral to positive reactions, particularly for moderate-sized cuts, consistent with relief effects. This regime-dependent pattern supports the signaling interpretation of short-horizon responses.

In [20]:
```
print("=== WEEK: Mean Excess Returns by Cut & Crisis ===")
print(week_tables["cut_crisis_mean"], "\n")

print("=== WEEK: Correlation with Cut Size ===")
print(week_tables["correlation"], "\n")
```

```
print("=== WEEK: Crisis vs Non-Crisis ===")
print(week_tables["crisis_summary"], "\n")
```

```
=== WEEK: Mean Excess Returns by Cut & Crisis ===
crisis          0         1
cut_bps
25       0.004500 -0.010914
50      -0.024469 -0.003669
75            NaN  0.054738
100           NaN -0.027679

=== WEEK: Correlation with Cut Size ===
          cut_bps    week_e
cut_bps  1.000000  0.027471
week_e   0.027471  1.000000

=== WEEK: Crisis vs Non-Crisis ===
            mean  count
crisis
0       0.001281      9
1      -0.002946     25
```

# Interpetation (Week)

The results show that market reactions to rate cuts are primarily front-loaded. On the announcement day, returns differ sharply between crisis and non-crisis regimes, consistent with signaling and relief effects. By one week out, these differences narrow substantially, and the correlation between cut size and returns falls close to zero, indicating that the market rapidly incorporates the policy information. This supports focusing on day-of and week-ahead windows rather than longer horizons when identifying sensitivity to rate-cut announcements.

In [21]:
```
print("=== MONTH: Mean Excess Returns by Cut & Crisis ===")
print(month_tables["cut_crisis_mean"], "\n")

print("=== MONTH: Correlation with Cut Size ===")
print(month_tables["correlation"], "\n")

print("=== MONTH: Crisis vs Non-Crisis ===")
print(month_tables["crisis_summary"], "\n")
```

```
=== MONTH: Mean Excess Returns by Cut & Crisis ===
crisis          0         1
cut_bps
25      -0.000283 -0.012827
50       0.031096 -0.024678
75            NaN  0.028495
100           NaN -0.068476

=== MONTH: Correlation with Cut Size ===
           cut_bps   month_e
cut_bps   1.000000 -0.238831
month_e -0.238831  1.000000

=== MONTH: Crisis vs Non-Crisis ===
             mean  count
crisis
0        0.003203      9
1       -0.020610     25
```

# Interpetation (Month)

One-month excess returns are meaningfully lower during crisis periods than during non-crisis periods, and larger rate cuts are associated with weaker average performance. The relationship between cut size and returns is negative, but inconsistent across cut magnitudes, with no clear monotonic pattern. At the monthly horizon, market behavior is dominated by broader economic and financial conditions rather than the rate-cut announcement itself. Differences in returns largely reflect the underlying state of the business cycle, making month-ahead results less suitable for isolating the direct impact of monetary policy actions.

# Running a OLS regression

We run an OLS regression to assess how equity returns respond to the size of rate cuts, whether the cut occurs during a crisis, and whether the impact of rate cuts differs across crisis and non-crisis regimes.

In [22]:
```
def run_ols(df, y_col):
    X = df[["cut_bps", "crisis"]].copy()
    X["cut_x_crisis"] = X["cut_bps"] * X["crisis"]
    X = sm.add_constant(X)

    y = df[y_col]
```

```
    model = sm.OLS(y, X).fit(cov_type="HC3")
    return model

ols_day   = run_ols(event_level, "day_e")
ols_week  = run_ols(event_level, "week_e")
ols_month = run_ols(event_level, "month_e")
```

In [23]:
```
print("=== DAY ===")
print(ols_day.summary())

print("\n=== WEEK ===")
print(ols_week.summary())

print("\n=== MONTH ===")
print(ols_month.summary())
```

```
=== DAY ===
                            OLS Regression Results
==============================================================================
=======
Dep. Variable:                  day_e   R-squared:
0.028
Model:                            OLS   Adj. R-squared:
-0.069
Method:                 Least Squares   F-statistic:
0.7439
Date:                Thu, 12 Feb 2026   Prob (F-statistic):
0.534
Time:                        23:48:20   Log-Likelihood:
90.529
No. Observations:                  34   AIC:
-173.1
Df Residuals:                      30   BIC:
-167.0
Df Model:                           3
Covariance Type:                  HC3
==============================================================================
=========
                 coef    std err          z      P>|z|      [0.025
0.975]
------------------------------------------------------------------------------
---------
const          -0.0091      0.010     -0.873      0.383      -0.029
0.011
cut_bps         0.0002      0.000      0.477      0.634      -0.000
0.001
crisis          0.0076      0.026      0.290      0.772      -0.044
0.059
cut_x_crisis   -0.0001      0.001     -0.149      0.881      -0.001
0.001
==============================================================================
=======
Omnibus:                       26.702   Durbin-Watson:
```

```
2.164
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
113.230
Skew:                          -1.342   Prob(JB):
2.59e-25
Kurtosis:                      11.528   Cond. No.
675.
================================================================================
=======

Notes:
[1] Standard Errors are heteroscedasticity robust (HC3)

=== WEEK ===
                            OLS Regression Results
================================================================================
=======
Dep. Variable:                  week_e   R-squared:
0.039
Model:                             OLS   Adj. R-squared:
-0.057
Method:                  Least Squares   F-statistic:
3.781
Date:                Thu, 12 Feb 2026   Prob (F-statistic):
0.0206
Time:                        23:48:20   Log-Likelihood:
71.856
No. Observations:                  34   AIC:
-135.7
Df Residuals:                      30   BIC:
-129.6
Df Model:                           3
Covariance Type:                  HC3
================================================================================
=========
                   coef    std err          z      P>|z|      [0.025
0.975]
--------------------------------------------------------------------------------
---------
const            0.0335      0.012      2.829      0.005       0.010
0.057
cut_bps         -0.0012      0.000     -3.361      0.001      -0.002
-0.000
crisis          -0.0448      0.037     -1.219      0.223      -0.117
0.027
cut_x_crisis     0.0013      0.001      1.465      0.143      -0.000
0.003
================================================================================
=======
Omnibus:                       19.676   Durbin-Watson:
1.676
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
```

```
36.976
Skew:                          -1.300   Prob(JB):
9.35e-09
Kurtosis:                       7.398   Cond. No.
675.
================================================================================
=======

Notes:
[1] Standard Errors are heteroscedasticity robust (HC3)

=== MONTH ===
                            OLS Regression Results
================================================================================
=======
Dep. Variable:                 month_e   R-squared:
0.119
Model:                             OLS   Adj. R-squared:
0.031
Method:                  Least Squares   F-statistic:
2.149
Date:                Thu, 12 Feb 2026   Prob (F-statistic):
0.115
Time:                        23:48:20   Log-Likelihood:
64.828
No. Observations:                  34   AIC:
-121.7
Df Residuals:                      30   BIC:
-115.5
Df Model:                           3
Covariance Type:                  HC3
================================================================================
=========
                   coef    std err          z      P>|z|      [0.025
0.975]
--------------------------------------------------------------------------------
---------
const           -0.0317      0.024     -1.306      0.192      -0.079
0.016
cut_bps          0.0013      0.001      1.410      0.159      -0.000
0.003
crisis           0.0281      0.034      0.828      0.407      -0.038
0.095
cut_x_crisis    -0.0016      0.001     -1.548      0.122      -0.004
0.000
================================================================================
=======
Omnibus:                       11.980   Durbin-Watson:
1.537
Prob(Omnibus):                  0.003   Jarque-Bera (JB):
12.220
Skew:                          -1.092   Prob(JB):
```

```
0.00222
Kurtosis:                       4.963   Cond. No.
675.
==============================================================================
=======

Notes:
[1] Standard Errors are heteroscedasticity robust (HC3)
```

# Results and Interpetation

## DAY

Result: No statistically significant relationship between rate cut size, crisis regime, or their interaction and same-day excess returns. The model explains very little variation in returns.

Interpretation: Same-day market reactions are noisy and dominated by immediate positioning and headline effects. While prices move on announcement day, those moves are not systematically explained by cut size or whether the cut occurs during a crisis.

## WEEK

Result: Week-ahead excess returns show a statistically significant negative relationship with cut size. Larger cuts are associated with weaker one-week performance, while crisis indicators and interaction terms are not individually significant.

Interpretation: Over a one-week horizon, investors appear to interpret larger rate cuts as stronger negative macro signals rather than supportive policy actions. This suggests that expectation and signaling effects persist beyond the announcement day but fade quickly thereafter.

## MONTH

Result: At the one-month horizon, coefficients are larger in magnitude but not statistically significant at conventional levels, and overall explanatory power increases modestly.

Interpretation: Monthly returns reflect broader macroeconomic conditions rather than the direct impact of rate-cut announcements, making this horizon less

suitable for isolating policy-driven market reactions.

# The intuition

The intuition behind the results reflects how markets process monetary policy information across different economic states. During crisis periods, investors are typically already positioned defensively, with elevated uncertainty and risk aversion largely reflected in equity prices. Rate cuts in these environments are often anticipated and perceived as reactive responses to deteriorating macroeconomic conditions rather than as new sources of support. Consequently, a rate-cut announcement tends to confirm existing concerns instead of delivering positive informational surprises, limiting its incremental impact on stock prices and, in some cases, contributing to weaker short-term performance.

In contrast, rate cuts during non-crisis periods are more likely to be interpreted as proactive policy adjustments rather than emergency interventions. In these settings, monetary easing can meaningfully affect discount rates, growth expectations, and risk premia, introducing new information for investors. Because expectations are less anchored to economic distress, policy actions in non-crisis environments carry greater informational content and are more likely to elicit a stronger equity market response. This state-dependent interpretation of monetary policy is consistent with the interaction effects observed in the regressions, indicating that the marginal impact of rate cuts on returns is significantly weaker during crisis periods.

# Seeing if this intuition is statiscally correct

In [24]:
```
def crisis_ttest(df, window):
    col = f"{window}_e"

    crisis_returns = df[df["crisis"] == 1][col]
    noncrisis_returns = df[df["crisis"] == 0][col]

    t_stat, p_val = stats.ttest_ind(
        crisis_returns,
        noncrisis_returns,
        equal_var=False   # Welch test
    )

    return {
        "window": window,
```

```
        "crisis_mean": crisis_returns.mean(),
        "noncrisis_mean": noncrisis_returns.mean(),
        "t_stat": t_stat,
        "p_value": p_val
    }

results = [
    crisis_ttest(event_level, "day"),
    crisis_ttest(event_level, "week"),
    crisis_ttest(event_level, "month")
]

pd.DataFrame(results)
```

Out[24]:

| | window | crisis_mean | noncrisis_mean | t_stat | p_value |
|---|---|---|---|---|---|
| **0** | day | 0.001266 | -0.004729 | 1.210311 | 0.235876 |
| **1** | week | -0.002946 | 0.001281 | -0.496616 | 0.623030 |
| **2** | month | -0.020610 | 0.003203 | -2.312003 | 0.027514 |

# Result and Interpetation

## DAY

Result: Average day-of excess returns are slightly higher during crisis periods than non-crisis periods, but the difference is not statistically significant.

Interpretation: Immediate market reactions to rate cuts do not differ meaningfully across regimes on the announcement day, consistent with high noise and rapid price adjustment.

## WEEK

Result: Week-ahead excess returns are marginally lower in crisis periods, but the difference is not statistically significant.

Interpretation: Any regime-based differences fade quickly after the announcement, suggesting limited persistence of the initial reaction.

## MONTH

Result: One-month excess returns are significantly lower during crisis periods

than during non-crisis periods.

Interpretation: At longer horizons, returns reflect broader macroeconomic conditions associated with crisis environments rather than the direct effect of the rate-cut announcement itself.

# Seeing what subsectors and equities are more effected

In [25]:
```
# Mapping each asset or index to a financial subsector
subsector_map = {
    # Banks
    "BKX": "Banks",
    "WFC": "Banks",
    "MS":  "Banks",

    # Insurance
    "KIE": "Insurance",
    "MKL": "Insurance",

    # Asset Management & Capital Markets
    "VFH": "Asset Mgmt & Capital Mkts",
    "AMP": "Asset Mgmt & Capital Mkts",
    "BX":  "Asset Mgmt & Capital Mkts",

    # Financial Technology
    "FINX": "Financial Technology",
    "V":    "Financial Technology",

    # Optional
    "O":   "Other",
    "PLD": "Other"
}
# Assigning subsector labels to each asset
event_panel_asset["subsector"] = event_panel_asset["asset"].map(subsec
event_panel_asset[["asset","subsector"]].drop_duplicates().sort_values

for window in ["day", "week", "month"]:
    print(f"\n=== {window.upper()}: Mean Excess Returns by Subsector =
    print(
        event_panel_asset
        .groupby("subsector")[f"{window}_e"]
        .mean()
        .sort_values(ascending=False)
    )


# Average excess return by subsector and asset
```

```
for window in ["day", "week", "month"]:
    print(f"\n=== {window.upper()}: Mean Excess Returns by Subsector =
    print(
        event_panel_asset
        .groupby("subsector")[f"{window}_e"]
        .mean()
        .sort_values(ascending=False)
    )


# Average excess return by subsector and crisis regime
for window in ["day", "week", "month"]:
    print(f"\n=== {window.upper()}: Subsector × Crisis ===")
    print(
        event_panel_asset
        .groupby(["subsector", "crisis"])[f"{window}_e"]
        .mean()
    )

# Finding how sensetive assets are to per bps cut
all_sensitivities = []

for window in ["day", "week", "month"]:
    for asset in event_panel_asset["asset"].unique():
        df = event_panel_asset[event_panel_asset["asset"] == asset]

        if len(df) >= 10:
            X = sm.add_constant(df["cut_bps"])
            y = df[f"{window}_e"]

            model = sm.OLS(y, X).fit(cov_type="HC3")

            all_sensitivities.append({
                "window": window,
                "asset": asset,
                "subsector": df["subsector"].iloc[0],
                "cut_sensitivity": model.params["cut_bps"],
                "t_stat": model.tvalues["cut_bps"],
                "p_value": model.pvalues["cut_bps"]
            })

sensitivities_df = pd.DataFrame(all_sensitivities)

# Ranking assets by sensitivity to rate cuts
pd.DataFrame(sensitivities).sort_values("cut_sensitivity", ascending=F
```

```
=== DAY: Mean Excess Returns by Subsector ===
subsector
Banks                          0.003521
Other                          0.000649
Financial Technology           0.000206
Insurance                     -0.001959
Asset Mgmt & Capital Mkts     -0.002447
```

```
Name: day_e, dtype: float64

=== WEEK: Mean Excess Returns by Subsector ===
subsector
Other                         0.003172
Financial Technology          0.001809
Banks                         0.001245
Insurance                    -0.000426
Asset Mgmt & Capital Mkts    -0.008415
Name: week_e, dtype: float64

=== MONTH: Mean Excess Returns by Subsector ===
subsector
Financial Technology         -0.002419
Other                        -0.005460
Insurance                    -0.005706
Banks                        -0.016823
Asset Mgmt & Capital Mkts    -0.026450
Name: month_e, dtype: float64

=== DAY: Mean Excess Returns by Subsector ===
subsector
Banks                         0.003521
Other                         0.000649
Financial Technology          0.000206
Insurance                    -0.001959
Asset Mgmt & Capital Mkts    -0.002447
Name: day_e, dtype: float64

=== WEEK: Mean Excess Returns by Subsector ===
subsector
Other                         0.003172
Financial Technology          0.001809
Banks                         0.001245
Insurance                    -0.000426
Asset Mgmt & Capital Mkts    -0.008415
Name: week_e, dtype: float64

=== MONTH: Mean Excess Returns by Subsector ===
subsector
Financial Technology         -0.002419
Other                        -0.005460
Insurance                    -0.005706
Banks                        -0.016823
Asset Mgmt & Capital Mkts    -0.026450
Name: month_e, dtype: float64

=== DAY: Subsector × Crisis ===
subsector                 crisis
Asset Mgmt & Capital Mkts  0       -0.006533
                           1       -0.000976
Banks                      0       -0.008150
```

```
                            1         0.007722
Financial Technology        0        -0.002528
                            1         0.001190
Insurance                   0        -0.001283
                            1        -0.002202
Other                       0        -0.003116
                            1         0.002005
Name: day_e, dtype: float64

=== WEEK: Subsector × Crisis ===
subsector                   crisis
Asset Mgmt & Capital Mkts   0        -0.004854
                            1        -0.009696
Banks                       0         0.005287
                            1        -0.000211
Financial Technology        0        -0.006536
                            1         0.004813
Insurance                   0         0.019126
                            1        -0.007464
Other                       0        -0.002831
                            1         0.005333
Name: week_e, dtype: float64

=== MONTH: Subsector × Crisis ===
subsector                   crisis
Asset Mgmt & Capital Mkts   0        -0.008302
                            1        -0.032983
Banks                       0         0.028217
                            1        -0.033037
Financial Technology        0        -0.011796
                            1         0.000957
Insurance                   0         0.006068
                            1        -0.009945
Other                       0         0.007869
                            1        -0.010258
Name: month_e, dtype: float64
```

Out[25]:

| | asset | subsector | cut_sensitivity |
|---|---|---|---|
| **5** | MS | Banks | 0.001083 |
| **7** | PLD | Other | 0.000447 |
| **0** | AMP | Asset Mgmt & Capital Mkts | 0.000149 |
| **2** | FINX | Financial Technology | 0.000035 |
| **3** | KIE | Insurance | -0.000272 |
| **8** | V | Financial Technology | -0.000469 |
| **9** | VFH | Asset Mgmt & Capital Mkts | -0.000562 |
| **4** | MKL | Insurance | -0.000650 |
| **1** | BX | Asset Mgmt & Capital Mkts | -0.000706 |
| **6** | O | Other | -0.001146 |
| **11** | ^BKX | NaN | -0.001370 |
| **10** | WFC | Banks | -0.001921 |

# Result and Interpetation

## Note: cut sensitivity measures how much a stock's timeframe return moves per 1 bp rate cut

## DAY

Result: Banks show the strongest positive day-of excess returns following rate cuts, while asset management and insurance subsectors post negative average reactions. In crisis periods, banks and financial technology outperform their non-crisis counterparts, whereas asset management underperforms in both regimes.

Interpretation: On the announcement day, market reactions are dominated by immediate signaling effects. Banks benefit the most in crisis settings, likely reflecting short-term relief and liquidity expectations, while asset managers and insurers face pressure as rate cuts signal weaker macro conditions that can hurt fee growth or underwriting margins.

## WEEK

Result: Financial technology and banks show modest positive average excess

returns over the following week, while asset management consistently underperforms. Crisis-period performance weakens for banks and insurance but improves for financial technology and other rate-insensitive sectors.

Interpretation: Over a one-week horizon, the market begins to differentiate across business models. Firms less reliant on net interest margins or balance-sheet leverage appear more resilient, while asset managers remain sensitive to expectations of slower growth and lower asset valuations.

## MONTH

Result: All subsectors exhibit negative average excess returns at the one-month horizon, with the largest declines concentrated in asset management and banks. Crisis periods are associated with substantially worse performance across nearly all subsectors.

Interpretation: At longer horizons, returns are driven primarily by broader macroeconomic conditions rather than the rate-cut announcement itself. Subsector differences largely reflect exposure to economic downturns and risk sentiment, reinforcing that month-ahead returns are less informative for isolating policy-driven effects.

# Summary

## Summary of Findings

Across all analyses, market reactions to rate cuts are highly time-dependent, with statistical significance concentrated in short horizons and diminishing as the window expands.

Day-of reactions show directional differences across subsectors and regimes, but these effects are not statistically significant in aggregate. Same-day returns are dominated by announcement noise, positioning, and headline effects, and neither cut size nor crisis regime meaningfully explains cross-sectional returns.

Week-ahead returns provide the clearest statistically significant evidence of market interpretation. Regression results indicate a significant negative relationship between cut size and one-week excess returns, suggesting that larger rate cuts are interpreted as stronger negative macroeconomic signals rather than supportive policy actions. While crisis indicators themselves are not independently significant, the week horizon captures persistent signaling effects

beyond the announcement day, making it the most informative window for identifying sensitivity to rate-cut announcements.

One-month returns show statistically significant differences between crisis and non-crisis periods, with crisis environments associated with substantially weaker performance. However, regression coefficients at this horizon are unstable and inconsistent across cut sizes and subsectors. This indicates that monthly outcomes are driven primarily by broader macroeconomic conditions rather than the rate-cut announcement itself, limiting their usefulness for isolating policy effects.

Subsector and asset-level results reinforce these conclusions. Balance-sheet-intensive subsectors such as banks and insurers exhibit greater sensitivity, particularly during crisis periods, while market-oriented firms show more resilience in short windows. However, heterogeneity within subsectors highlights the importance of firm-specific exposure over broad classification.

Overall, statistical significance is strongest at the one-week horizon, where rate cuts carry the clearest informational content. Day-of reactions are noisy and difficult to attribute, while month-ahead returns are contaminated by macroeconomic dynamics. As a result, day and week windows are the most appropriate for evaluating the market impact of rate-cut announcements.

## Crisis vs. Non-Crisis Regimes

The market response to rate cuts differs meaningfully depending on whether the economy is in a crisis or non-crisis regime. In crisis periods, rate cuts are generally anticipated and interpreted as reactive measures to worsening economic conditions. As a result, announcement-day and week-ahead reactions tend to be muted, and one-month returns are significantly weaker. Statistical tests confirm that crisis periods are associated with materially lower one-month excess returns, reflecting the dominant influence of macroeconomic stress rather than the direct effect of policy actions.

In contrast, during non-crisis periods, rate cuts are more likely to be viewed as proactive adjustments. While same-day reactions remain noisy, week-ahead performance is more stable, and one-month returns are modestly positive on average. These patterns suggest that the informational content of monetary easing is stronger outside of crisis environments, where expectations are less anchored to economic distress.

# Impact by Size of Rate Cut (Basis Points)

The market's interpretation of rate cuts also varies by the magnitude of the policy move. Smaller cuts (25 bps) generally produce limited and inconsistent responses across all time horizons, indicating that they are often viewed as fine-tuning rather than meaningful shifts in policy stance.

Moderate cuts (50–75 bps) generate the strongest and most informative reactions, particularly over the one-week horizon. Regression results show that larger cuts within this range are associated with weaker short-term returns, consistent with the view that the market interprets these moves as signals of deteriorating economic conditions rather than immediate support.

Very large cuts (100 bps) are consistently associated with negative performance, especially at longer horizons. These actions tend to occur in severe stress environments and are interpreted as confirmation of significant macroeconomic weakness. As a result, their observed impact reflects underlying economic conditions rather than the mechanical effect of lower interest rates.